Минимален скелет на граф

```
# Максимален брой върхове в графа
MAXN ≔ 150
MAX_VALUE ≔ 10000

# Брой върхове в графа
n ≔ 9

# Матрица на теглата на графа
A ≔ [
   [0, 1, 0, 2, 0, 0, 0, 0, 0],
   [1, 0, 3, 0, 13, 0, 0, 0, 0],
   [0, 3, 0, 4, 0, 3, 0, 0, 0],
   [2, 0, 4, 0, 0, 16, 14, 0, 0],
   [0, 13, 0, 0, 0, 12, 0, 1, 13],
   [0, 0, 3, 16, 12, 0, 1, 0, 1],
   [0, 0, 0, 14, 0, 1, 0, 0, 0],
   [0, 0, 0, 0, 1, 0, 0, 0, 0],
   [0, 0, 0, 0, 13, 1, 0, 0, 0]
]

used ≔ масив[MAXN] от 0
prev ≔ масив[MAXN] от 0
T ≔ масив[MAXN] от 0

функция prim():
   MST ≔ 0  # тук трупаме цената на минималното покриващо дърво
   # инициализация
```

```
    за i от 0 до n прави:
        used[i] ≔ 0
        prev[i] ≔ 0
    used[0] ≔ 1  # избираме произволен начален връх
    за i от 0 до n прави:
        T[i] ≔ A[0][i] ако A[0][i] то иначе MAX_VALUE
    за _ от 0 до n – 1 прави:
        # търсене на следващо минимално ребро
        minDist ≔ MAX_VALUE
        j ≔ -1
        за i от 0 до n прави:
            ако не used[i] и T[i] < minDist то:
                minDist ≔ T[i]
                j ≔ i
        used[j] ≔ 1
        print(f"({prev[j] + 1},{j + 1}) ", end="")
        MST ≔ MST + minDist
        за i от 0 до n прави:
            ако не used[i] и A[j][i] и T[i] > A[j][i] то:
                T[i] ≔ A[j][i]
                # запазване на предшественика
                # за евентуално отпечатване на следващо минимално ребро
                prev[i] ≔ j
    отпечатай(f"\nЦената на минималното покриващо дърво е {MST}.\n")

prim()
```